

Binary Search Hardware Performance Analysis

Objective

Problem Statement

Analyze Binary Search performance under different hardware architectures by simulating variations in processor speed. Record execution time and compare results. Interpret how hardware factors influence algorithm efficiency. Justify its robustness across systems.

Algorithm

- Create a sorted array containing 1,000,000 elements.
- Implement Binary Search to locate the target element (987654).
- Simulate different processor speeds using artificial delays (0.001 s, 0.005 s, 0.01 s).
- Measure execution time using `time.perf_counter()`.
- Compare execution times for all simulated processors.
- Interpret the effect of processor speed while noting Binary Search complexity remains $O(\log n)$.

Python Concepts Used

Functions, loops, conditional statements, Binary Search, `time.sleep()`, `time.perf_counter()`, lists, formatted output.

Input

Sorted array: 1 to 1,000,000

Target element: 987654

Expected Output

High-Speed Processor (3.5 GHz): ~0.0011 s

Medium-Speed Processor (2.0 GHz): ~0.0051 s

Low-Speed Processor (1.0 GHz): ~0.0101 s

Target found at index 987653.

Execution Screenshot

The screenshot shows the Programiz Python Online Compiler interface. On the left, a file explorer shows 'main.py'. The code editor contains a Python script for a binary search algorithm. The script defines a target value (987654), a delay, and a list of simulated processors (High-Speed, Medium-Speed, Low-Speed). It then performs a binary search for the target in the list and prints the execution time for each processor. The output window on the right shows the results of the execution, including the target found at index 987653 and the execution times for each processor. A 'Performance Comparison' section summarizes the results, and a 'Conclusion' section states that Binary Search is robust across different systems. A 'Code Execution Successful' message is also displayed.

```
main.py
33 target = 987654
34
35 start = time.perf_counter()
36
37 # Artificial CPU Delay
38 time.sleep(delay)
39
40 index = binary_search(arr, target)
41
42 end = time.perf_counter()
43
44 execution_time = end - start
45
46 if index != -1:
47     print("Target Found at Index:", index)
48 else:
49     print("Target Not Found")
50
51 print("Execution Time: {:.6f} seconds".format(execution_time))
52
53 return execution_time
54
55 # Simulated Hardware Architectures
56 fast = simulate_processor("High-Speed Processor (3.5 GHz)", 0.001)
57 medium = simulate_processor("Medium-Speed Processor (2.0 GHz)", 0.005)
58 slow = simulate_processor("Low-Speed Processor (1.0 GHz)", 0.01)
59
60
61 print("\n----- Performance Comparison -----")
62 print("Fast Processor : ", round(fast, 6), "seconds")
63 print("Medium Processor : ", round(medium, 6), "seconds")
64 print("Slow Processor : ", round(slow, 6), "seconds")
65
66 print("\nConclusion:")
67 print("1. Faster processors execute Binary Search more quickly.")
68 print("2. Slower processors increase execution time.")
69 print("3. The number of Binary Search comparisons remains almost the same.")
70 print("4. Binary Search keeps O(log n) time complexity regardless of hardware.")
71 print("5. This demonstrates that Binary Search is robust across different systems")
```

Output

```
Processor: High-Speed Processor (3.5 GHz)
Artificial Delay: 0.001 seconds
-----
Target Found at Index: 987653
Execution Time: 0.001125 seconds
-----
Processor: Medium-Speed Processor (2.0 GHz)
Artificial Delay: 0.005 seconds
-----
Target Found at Index: 987653
Execution Time: 0.005130 seconds
-----
Processor: Low-Speed Processor (1.0 GHz)
Artificial Delay: 0.01 seconds
-----
Target Found at Index: 987653
Execution Time: 0.010141 seconds
-----
Performance Comparison -----
Fast Processor : 0.001125 seconds
Medium Processor : 0.005130 seconds
Slow Processor : 0.010141 seconds
Conclusion:
1. Faster processors execute Binary Search more quickly.
2. Slower processors increase execution time.
3. The number of Binary Search comparisons remains almost the same.
4. Binary Search keeps O(log n) time complexity regardless of hardware.
5. This demonstrates that Binary Search is robust across different systems.
==== Code Execution Successful ====
```

Observation Table

Processor	Artificial Delay	Execution Time
High-Speed (3.5 GHz)	0.001 s	0.001125 s
Medium-Speed (2.0 GHz)	0.005 s	0.005130 s
Low-Speed (1.0 GHz)	0.010 s	0.010141 s

Analysis

The measured execution time increases as the simulated processor becomes slower. Although the wall-clock time changes, Binary Search performs the same sequence of comparisons because the algorithm depends on input size rather than processor frequency. Faster hardware reduces execution time but does not change the algorithmic complexity.

Hardware Influence

- Processor speed affects elapsed execution time.
- Memory hierarchy and cache can further improve practical performance.
- Binary Search benefits from efficient memory access because it performs very few comparisons.
- Algorithm efficiency is primarily determined by $O(\log n)$, not hardware alone.

Time Complexity

Best Case: $O(1)$

Average Case: $O(\log n)$

Worst Case: $O(\log n)$

Space Complexity: $O(1)$

Conclusion

The experiment demonstrates that Binary Search is highly robust across different hardware systems. Faster processors reduce execution time, while slower processors increase elapsed time. However, the logarithmic time complexity remains unchanged, proving that Binary Search scales efficiently and maintains excellent performance regardless of the underlying hardware architecture.